

Data loader

July 9, 2019

```
In [1]: import torch
import torch.nn as nn
import torch.nn.functional as f
from torch.autograd import Variable as V

In [2]: from torch.utils.data import Dataset, DataLoader

In [3]: import pandas as pd
import numpy as np
```

0.0.1 Get Dataset

```
In [4]: #dataset = pd.read_pickle('dataset.pkl')

In [5]: #dataset = dataset.values

In [6]: dataset = np.load('dataset_s112.npy')

In [7]: dataset = np.array([i.reshape(1,112,112) for i in dataset])

In [8]: dataset.shape

Out[8]: (10000, 1, 112, 112)
```

0.0.2 Split Data Train/Val

```
In [9]: def split(dataset,prc=0.2):
    size = len(dataset)
    size = int(size*prc)
    val = dataset[:size]
    trn = dataset[size:]
    return trn , val

In [10]: trn , val = split(dataset)
```

0.0.3 Prepare DataLoaders

```
In [11]: trn_dl = DataLoader(trn,batch_size=32,num_workers=2,shuffle=False)
val_dl = DataLoader(val,batch_size=32,num_workers=2,shuffle=False)
```

0.1 Model

```
In [127]: class o_to_o(nn.Module):
          def __init__(self):
              super(o_to_o,self).__init__()
              self.conv1 = nn.Conv2d(1,1,(8,14))
              self.pool = nn.MaxPool2d((30,30),stride=1)

              self.deconv1 =nn.ConvTranspose2d(1,1,(30,30))
              self.deconv2 = nn.ConvTranspose2d(1,1,(8,14))

          def forward(self,inputs):

              k = f.elu(self.conv1(inputs))

              k = f.elu(self.pool(k))

              k = f.elu(self.deconv1(k))

              k = torch.sigmoid(self.deconv2(k))

              return k

In [130]: m = o_to_o().cuda()
          opt = torch.optim.Adam(m.parameters(),1e-3)

In [131]: def fit(epoch):
          for j in range(epoch):
              train_loss = 0
              val_loss = 0
              for i , data in enumerate(trn_dl):
                  opt.zero_grad()
                  #print(data.requires_grad)
                  data = data.type(torch.float).cuda()
                  out = m(V(data))
                  #data = data.type(torch.LongTensor).cuda()
                  #print(out.requires_grad,data.requires_grad)
                  loss = f.binary_cross_entropy(out,data)
                  loss.backward()
                  opt.step()

                  train_loss += loss.data

              for i , data in enumerate(val_dl):

                  data = data.type(torch.float).cuda()
                  out = m(V(data))
                  loss = f.binary_cross_entropy(out,data)
                  loss.backward()
```

```

        opt.step()

        val_loss += loss

        print('train Loss: ', train_loss.tolist(), ' validation loss: ', val_loss.tolist())

In [132]: fit(3)

train Loss:  51.46788787841797  validation loss:  9.5443115234375
train Loss:  28.031633377075195  validation loss:  6.640884876251221
train Loss:  24.30136489868164  validation loss:  8.332087516784668

In [133]: for i in opt.param_groups:
            i['lr'] = 1e-4

In [134]: fit(3)

train Loss:  41.61981964111328  validation loss:  8.855514526367188
train Loss:  30.667396545410156  validation loss:  7.1804351806640625
train Loss:  26.34232521057129  validation loss:  6.335571765899658

```